

Mongo

1. Realizar las operaciones necesarias para poder ejecutar las siguientes consultas de forma eficiente. Como máximo 4 consultas. Pueden ser necesarias menos de 4.

db.zips.createIndex({state: "text"})

db.zips.createIndex({population: 1})

db.zips.createIndex({location: "2dsphere" })

2. Estados (state) cuya población media sea mayor que 15000 habitantes a excepción del estado de California (CA) y Florida (FL).

db.zips.aggregate()

3. Listado de las ciudades (city) agrupadas por estado (state) que estén a menos de 15km de las coordenadas con longitud -72.62 y latitud 42.07

db.zips.aggregate()

Redis

3. Devolver las reproducciones totales de todas las canciones realizadas por los dos usuarios. (1 consulta)

ZUNION 2 Conchi Adolfo WITHSCORES

4. Crear un nuevo listado llamado "Conchi-Adolfo" de las canciones que han escuchado tanto "Adolfo" como "Conchi" con el valor de reproducciones más bajo realizado por cualquiera de los dos usuarios. (1 consulta, no se pide mostrarlo)

2. Indicar que la usuaria "Conchi" ha reproducido una vez más la canción "Amor". (1 consulta)

ZINCRBY Conchi 1 Amor

1. Inicializar la BBDD con las reproducciones para dos usuarios:

a. La usuaria "Conchi" ha reproducido las canciones "Lola" 2 veces, y "Amor" 6 veces. (1 consulta)

ZADD "Conchi" 2 "Lola" 6 "Amor"

b. El usuario "Adolfo" ha reproducido las canciones, "Adios" 30 veces, "Dolor" 2 veces y "Amor" 15 veces. (1 consulta)

ZADD "Adolfo" 30 "Adios" 2 "Dolor" 15 "Amor"

2. Crear una relación etiquetada como "road" en una sola dirección (cualquiera) entre cada dos ciudades diferentes. No puede haber relaciones de las ciudades consigo mismas. (una consulta)

1. Crear cuatro nodos de tipo **city** con un atributo llamado **type** que siempre tendrá el valor "**province**" y otro atributo **name** con los siguientes valores: "**Madrid**", "**Valencia**", "**Sevilla**" y "**Barcelona**". (una consulta)

3. Borrar todos los atributos "**type**" de todos los nodos la base de datos (una consulta).

Cassandra

Selecciona la respuesta VERDADERA sobre la arquitectura Read Path Flow:

1.

Key Cache es una caché que permite almacenar resultados de las últimas consultas realizadas.

2.

La Row Caché está activada por defecto.

3.

Partition Index contiene el conjunto completo de posiciones para cada partition key en su SSTable asociada en la memoria.

4.

Partition Summary es un índice del Partition Index que se encuentra almacenado en el disco duro.

5.

Ninguna de las anteriores es verdadera.

Cuál de los siguientes trayectos es el más rápido durante la lectura de datos en la arquitectura de Read Path Flow:

1.

Desde Bloom Filter, pasando por Key Caché, pasando por Partition Summary, pasando por Partition Index y terminando en la posición correcta de la SSTable.

2.

Directamente accediendo a Key Caché ignorando el Bloom Filter y sin entrar en la SSTable.

3.

Comenzando en el Bloom Filter y pasando por Key Caché para acceder a la SSTable.

4.

Desde el Bloom Filter saltando directamente al Partition Index y después accediendo a la SSTable.

5.

Desde el Bloom Filter directamente accediendo al SSTable.

Indica que respuesta es correcta sobre el proceso de lectura de datos según la arquitectura del Read Path Flow:

1.

Los datos solo pueden leerse del SSTable.

2.

Los datos siempre se leen del Bloom Filter o Row Cache si está activada.

3.

Los datos solo pueden leerse del SSTable o Partition Summary.

4.

Los datos pueden leerse del Row Cache o de Key Cache o de SSTable.

5.

Los datos pueden leerse o del Row Cache, de SSTable o Memtable.